

Respuestas teóricas - Prueba técnica Full Stack

Andrés Felipe García

Índice

Python	2
1. ¿Qué ventajas ofrece Python para el desarrollo backend en comparación con otros lenguajes?	2
2. ¿Cómo funciona el manejo de memoria en Python y qué impacto tiene en aplicaciones web?	2
3. ¿Qué diferencias existen entre programación orientada a objetos y programación funcional en Python?	2
4. ¿Qué es un decorador en Python y en qué casos se utiliza en aplicaciones web?	2
5. ¿Cómo manejarías concurrencia en Python (threads, multiprocessing, async)?	3
6. ¿Qué buenas prácticas aplicarías para estructurar un proyecto Python productivo?	3
7. ¿Cómo manejarías excepciones de forma centralizada en una aplicación Python?	3
8. ¿Cómo implementarías logging en Python para entornos productivos?	3
Django y Django REST Framework	3
9. ¿Qué es Django y cuáles son sus componentes principales dentro de su arquitectura?	3
10. ¿Cómo funciona el patrón MTV (Model-Template-View) en Django?	4
11. ¿Cómo estructurarías un proyecto Django para que sea escalable y mantenible?	4
12. ¿Cómo implementarías validaciones de datos en modelos y serializers?	4
13. ¿Cómo diseñarías una API REST utilizando Django REST Framework?	4
14. ¿Cómo manejarías autenticación y autorización en Django?	4
15. ¿Qué estrategias usarías para manejar permisos a nivel de vista o endpoint?	5
Arquitectura de software	5
16. ¿Cuáles son las diferencias principales entre un monolito y una arquitectura de microservicios?	5
17. ¿Cuándo usarías comunicación síncrona y cuándo asíncrona entre servicios?	5
18. ¿Qué es un API Gateway y qué funciones cumple?	5
19. ¿Qué es el patrón circuit breaker y por qué es importante?	5
20. ¿Cómo implementarías observabilidad en un sistema distribuido?	6
21. ¿Cómo diseñarías el versionamiento de APIs REST en producción?	6
Angular	6
22. ¿Cuál es la diferencia entre un componente y un servicio en Angular?	6
23. ¿Qué es RxJS y cómo se utiliza en Angular?	6
24. ¿Cómo manejarías el estado de una aplicación Angular en un proyecto empresarial?	6
25. ¿Para qué sirven los guards en Angular?	7
26. ¿Cómo funcionan los formularios reactivos y cuándo usarlos?	7
27. ¿Qué son los interceptores HTTP y qué casos de uso tienen?	7
28. ¿Cómo estructurarías una arquitectura Angular escalable para múltiples módulos?	7

Estas respuestas resumen los conceptos solicitados y muestran cómo se relacionan con el sistema de rutas.

Python

1. ¿Qué ventajas ofrece Python para el desarrollo backend en comparación con otros lenguajes?

Respuesta. Python permite entregar funcionalidad backend con rapidez gracias a su sintaxis legible, tipado dinámico, biblioteca estándar amplia y ecosistema maduro. Frameworks como Django incluyen ORM, seguridad, migraciones, administración y convenciones listas para usar. La contrapartida es que, para trabajo intensivo de CPU o latencias extremadamente bajas, lenguajes compilados pueden rendir mejor; por eso la elección debe partir del problema y no de una preferencia absoluta.

Aplicación al proyecto. Django y pandas/openpyxl permitieron modelar el dominio, importar seis hojas de Excel, normalizar JSON, validar 5.000 rutas y exponer la API con poco código de infraestructura. PostgreSQL realiza las consultas relacionales y espaciales aproximadas; Angular se ocupa de la experiencia de usuario. Es una división pragmática de responsabilidades.

2. ¿Cómo funciona el manejo de memoria en Python y qué impacto tiene en aplicaciones web?

Respuesta. El modelo depende de la implementación. CPython usa principalmente conteo de referencias y añade un recolector para ciclos. Además, su asignador puede conservar bloques para reutilizarlos, por lo que liberar objetos no implica que el RSS del proceso baje inmediatamente. En una aplicación web esto obliga a vigilar objetos grandes, ciclos, cachés sin límite y recursos externos, y a configurar límites y reciclaje de workers como protección complementaria, no como sustituto de corregir fugas.

Aplicación al proyecto. La importación debe limitar tamaño y tipo de archivo, recorrer filas de forma controlada y guardar por lotes dentro de una estrategia transaccional. No mantendría simultáneamente el workbook completo, todas las entidades ORM y todos los errores serializados si el volumen creciera mucho. El endpoint devuelve solo una muestra de incidencias y deja el conjunto completo persistido.

3. ¿Qué diferencias existen entre programación orientada a objetos y programación funcional en Python?

Respuesta. La orientación a objetos organiza comportamiento y estado alrededor de objetos que colaboran; la programación funcional enfatiza funciones, composición, datos inmutables y reducción de efectos secundarios. Python es multiparadigma: usaría objetos para conceptos del dominio y contratos con ciclo de vida, y funciones puras para transformaciones y reglas fáciles de probar. No son enfoques excluyentes.

Aplicación al proyecto. Los modelos Django representan entidades persistentes, mientras las reglas de normalización y validación del importador se implementan como funciones puras cuando es posible. Así pueden probarse sin levantar Django ni PostgreSQL; el perfilador independiente reutiliza la misma semántica.

4. ¿Qué es un decorador en Python y en qué casos se utiliza en aplicaciones web?

Respuesta. Un decorador es un callable que recibe otro callable o clase y devuelve una versión enriquecida o reemplazada, normalmente mediante la sintaxis `@decorador`. Sirve para aplicar preocupaciones transversales -autorización, transacciones, caché, métricas o registro- sin repetirlas en cada función. Debe preservar metadatos con `functools.wraps` y no ocultar reglas de negocio inesperadas.

Aplicación al proyecto. Podría medirse la duración de la importación con un decorador, aunque para una vista DRF preferiría middleware o instrumentación para correlacionar toda la solicitud. La atomicidad se expresa claramente con `transaction.atomic`.

5. ¿Cómo manejarías concurrencia en Python (threads, multiprocessing, async)?

Respuesta. Primero revisaría el tipo de trabajo. Para operaciones de entrada y salida bloqueantes usaría threads; para cálculos intensivos de CPU, procesos; y para muchas operaciones de entrada y salida concurrentes con librerías compatibles, asyncio. Para tareas largas o con reintentos usaría una cola de trabajo. La elección debe basarse en mediciones y en las necesidades reales del sistema.

Aplicación al proyecto. Con 5.000 filas, la importación síncrona es defendible para el MVP. Si aumentara el volumen, POST /imports crearía un lote PENDING, una tarea procesaría el archivo por chunks y el cliente consultaría el estado. Evitaría ejecutar dos veces un mismo lote mediante hash o clave idempotente.

6. ¿Qué buenas prácticas aplicarías para estructurar un proyecto Python productivo?

Respuesta. Separaría configuración, dominio, casos de uso e infraestructura; agruparía código por capacidad de negocio; declararía dependencias y entornos reproducibles; aplicaría formato, lint, tipos y pruebas en CI; y mantendría secretos fuera del repositorio. La estructura debe ayudar a cambiar y probar el sistema, no añadir capas ceremoniales.

Aplicación al proyecto. El repositorio separa backend, frontend, SQL, Postman, datos y documentación. Dentro del backend, la lógica de importación no debería vivir en una vista enorme: la vista valida el transporte, un servicio coordina y las reglas puras validan filas.

7. ¿Cómo manejarías excepciones de forma centralizada en una aplicación Python?

Respuesta. Definiría excepciones de dominio con significado, las convertiría a respuestas HTTP en un único borde y registraría una vez la información operativa. Los errores esperados producirían códigos estables y mensajes seguros; los inesperados devolverían un 500 genérico con un identificador de correlación, conservando el traceback solo en logs. No usaría except Exception: pass.

Aplicación al proyecto. Los errores de cada fila son datos esperados de negocio y se acumulan como ImportIssue; una corrupción del workbook o un fallo de base de datos es una excepción del proceso. Separarlos permite completar un lote con rechazos válidos sin esconder fallos sistémicos.

8. ¿Cómo implementarías logging en Python para entornos productivos?

Respuesta. Usaría logging con configuración central por ambiente, salida estructurada a stdout, niveles coherentes e identificadores de correlación. La aplicación registra eventos; la plataforma recolecta, retiene y consulta. Evitaría información sensible, controlaría volumen y conectaría errores a métricas y trazas.

Aplicación al proyecto. Registraría inicio y fin de importación, conteos, duración y fallo sistémico con batch_id; no emitiría 4.342 stack traces por rechazos de datos. RouteExecutionLog conserva la auditoría funcional y el logging técnico explica el comportamiento del servicio.

Django y Django REST Framework

9. ¿Qué es Django y cuáles son sus componentes principales dentro de su arquitectura?

Respuesta. Django es un framework web de Python orientado a construir aplicaciones completas con convenciones y componentes integrados. Sus piezas principales incluyen ORM y modelos, enrutamiento, vistas, templates, formularios, middleware, autenticación, administración, migraciones, caché y herramientas de seguridad. Su integración reduce decisiones repetitivas, pero cada componente puede sustituirse o no utilizarse.

Aplicación al proyecto. Usamos modelos y migraciones para rutas, lotes, incidencias y logs; URLconf y DRF para /api/v1; middleware/configuración para seguridad; y PostgreSQL como persistencia. No usamos templates para la interfaz porque Angular consume la API.

10. ¿Cómo funciona el patrón MTV (Model-Template-View) en Django?

Respuesta. En el patrón MTV, el Model representa los datos y sus reglas, el Template se encarga de la presentación y la View recibe la solicitud, ejecuta el caso de uso y construye la respuesta. El URLconf determina qué vista debe atender cada ruta. Es parecido a MVC, aunque Django distribuye algunas responsabilidades con nombres diferentes.

Aplicación al proyecto. POST /api/v1/routes/import/ llega a una vista DRF, valida el archivo mediante un serializer, llama al servicio importador y persiste mediante modelos. La respuesta se serializa como JSON. Angular es la capa de presentación separada, por lo que el Template tradicional de Django no participa.

11. ¿Cómo estructurarías un proyecto Django para que sea escalable y mantenible?

Respuesta. Lo dividiría en apps cohesionadas por capacidad de negocio, no por tipo técnico global. Mantendría settings por ambiente, APIs versionadas, servicios para casos de uso, selectores o repositorios solo cuando aporten claridad, y pruebas junto a cada app. Evitaría que vistas, signals o modelos se conviertan en contenedores de toda la lógica.

Aplicación al proyecto. El dominio puede permanecer en apps.logistics mientras su cohesión sea alta. Si crecieran planificación, facturación y flota con equipos o ciclos distintos, los separaría primero como módulos internos bien delimitados. Solo después evaluaría despliegues independientes.

12. ¿Cómo implementarías validaciones de datos en modelos y serializers?

Respuesta. Validaría en varias capas porque protegen cosas diferentes. La base de datos asegura invariantes universales con tipos, NOT NULL, UNIQUE, claves foráneas y CHECK; el modelo expresa reglas reutilizables del dominio; el serializer valida el contrato de una operación y relaciones entre campos. La interfaz también puede ayudar, pero nunca es la autoridad.

Aplicación al proyecto. El importador valida forma, fechas, distancia, coordenadas, direcciones, catálogo y coherencia de ciudad por fila; conserva incidencias detalladas y persiste solo rutas válidas. El endpoint de creación reutiliza reglas equivalentes y la base asegura claves y restricciones estructurales.

13. ¿Cómo diseñarías una API REST utilizando Django REST Framework?

Respuesta. Empezaría por recursos y casos de uso, definiría contratos HTTP coherentes y después elegiría serializers, views y routers de DRF. Usaría sustantivos en URLs, verbos HTTP y status codes correctos, validación explícita, paginación y filtros. Añadiría autenticación, permisos, throttling, documentación OpenAPI, pruebas e idempotencia donde haya reintentos.

Aplicación al proyecto. La API está bajo /api/v1, lista rutas con paginación, filtros, búsqueda y orden; importa por multipart; crea mediante payload validado; ejecuta una selección de forma segura; y permite consultar logs. La colección Postman documenta ejemplos ejecutables y asserts.

14. ¿Cómo manejarías autenticación y autorización en Django?

Respuesta. Separaría identidad de permisos. Autenticación determina quién realiza la petición; autorización decide qué puede hacer sobre qué recurso. Para una aplicación web del mismo origen usaría sesiones con CSRF; para SPA o clientes externos, un proveedor OIDC/OAuth2 o tokens de vida corta con rotación. En todos los casos aplicaría HTTPS, hashing de Django, mínimo privilegio y validación server-side.

Aplicación al proyecto. Para un panel interno, una primera opción sólida es sesión Django si frontend y API comparten dominio. Si Angular se despliega separado o existen integraciones, usaría OIDC con access tokens cortos. Los roles podrían distinguir consulta, importación y ejecución.

15. ¿Qué estrategias usarías para manejar permisos a nivel de vista o endpoint?

Respuesta. Establecería una política segura global -normalmente usuario autenticado- y abriría excepciones de manera explícita. Combinaría permisos por rol o capacidad a nivel de vista con permisos por objeto para propiedad o alcance. Además filtraría el queryset, porque comprobar un objeto solo después de recuperarlo puede filtrar existencia o datos.

Aplicación al proyecto. Un observador podría listar y consultar; un operador importar y ejecutar; un administrador revisar incidencias. Cada consulta se limitaría a la organización del usuario y cada ejecución comprobaría estado y alcance dentro de una transacción.

Arquitectura de software

16. ¿Cuáles son las diferencias principales entre un monolito y una arquitectura de microservicios?

Respuesta. Un monolito se despliega como una sola aplicación y puede organizarse internamente en módulos. Los microservicios dividen el sistema en servicios independientes que pueden desplegarse y escalar por separado. Esa independencia también aumenta la complejidad de comunicación, datos, monitoreo y manejo de fallos. La decisión depende del tamaño del producto, del equipo y de la necesidad real de independencia.

Aplicación al proyecto. Elegiría un monolito modular Django: rutas, importación y ejecución comparten datos, transacciones y un equipo pequeño. Separaría la importación como worker, no necesariamente como microservicio. Solo extraería una capacidad si necesita escalar, desplegarse o fallar de forma independiente.

17. ¿Cuándo usarías comunicación síncrona y cuándo asíncrona entre servicios?

Respuesta. Usaría síncrona cuando el solicitante necesita una respuesta inmediata para continuar y la dependencia puede cumplir un presupuesto de latencia y disponibilidad. Usaría asíncrona cuando el trabajo puede diferirse, conviene desacoplar picos, hacer fan-out, tolerar indisponibilidad temporal o ejecutar procesos largos. Muchas soluciones combinan comando síncrono corto con procesamiento y eventos asíncronos.

Aplicación al proyecto. Consultar el detalle de una ruta es síncrono. Una importación grande debería crear un lote y procesarse asíncronamente; el cliente consulta estado o recibe notificación. La ejecución masiva puede dividirse en comandos idempotentes y publicar resultados.

18. ¿Qué es un API Gateway y qué funciones cumple?

Respuesta. Un API Gateway es el punto de entrada administrado para clientes frente a varios servicios. Enruta peticiones y centraliza preocupaciones transversales como terminación TLS, autenticación, rate limiting, observabilidad y, con cuidado, agregación de respuestas. No debe convertirse en un monolito con lógica de negocio ni en un punto único de fallo sin alta disponibilidad.

Aplicación al proyecto. En el MVP, un reverse proxy puede exponer Angular y /api/v1 con TLS y CORS simplificado. Si surgieran servicios de flota, notificaciones e identidad, el gateway enrutaría y aplicaría políticas comunes, mientras Django seguiría comprobando permisos de ejecución.

19. ¿Qué es el patrón circuit breaker y por qué es importante?

Respuesta. Circuit breaker protege las llamadas a un servicio que está fallando o respondiendo muy lento. Normalmente deja pasar las solicitudes mientras el servicio funciona; si supera un límite de errores, abre el circuito y falla rápidamente. Después permite algunas pruebas para verificar si el servicio se recuperó. Esto evita consumir recursos innecesariamente y reduce fallos en cadena.

Aplicación al proyecto. Si ejecutar una ruta invocara un proveedor externo de despacho, aplicaría timeout estricto y breaker. Con el circuito abierto registraría el intento como fallo temporal y respondería 503/estado reintentable; no marcaría la ruta como ejecutada.

20. ¿Cómo implementarías observabilidad en un sistema distribuido?

Respuesta. Instrumentaría logs, métricas y trazas correlacionadas, partiendo de objetivos de servicio y recorridos críticos. Propagaría `trace_id/request_id` entre HTTP, colas y tareas; mediría latencia, tráfico, errores y saturación; y construiría dashboards y alertas accionables sobre síntomas. OpenTelemetry permite instrumentación neutral respecto al backend de almacenamiento.

Aplicación al proyecto. Mediría duración y resultado de importaciones, filas/segundo, tasa de rechazo por código, latencia p95 de listados, ejecuciones fallidas y pool de PostgreSQL. Un `batch_id` uniría request, logs y registros; una traza mostraría API → importador → base.

21. ¿Cómo diseñarías el versionamiento de APIs REST en producción?

Respuesta. Versionaría el contrato externo solo ante cambios incompatibles, normalmente con una versión mayor en la URL como `/api/v1`. Los cambios aditivos deberían conservar la versión. Mantendría v1 y v2 en paralelo durante una migración, con documentación, métricas de adopción, fecha de retiro y pruebas de contrato. La compatibilidad vale más que incrementar versiones por cada release interno.

Aplicación al proyecto. `/api/v1` fija el contrato actual. Si v2 cambia la forma de los errores o el modelo de ejecución, mantendría serializers y rutas v1 mientras ambos llaman al mismo dominio cuando sea posible. Agregar un filtro opcional no exige v2.

Angular

22. ¿Cuál es la diferencia entre un componente y un servicio en Angular?

Respuesta. Un componente controla una parte de la interfaz: tiene clase, template, selector y estilos, recibe entradas y emite eventos. Un servicio encapsula funcionalidad reutilizable, acceso a datos, estado o lógica que no depende de una vista y se consume mediante inyección de dependencias. Mantengo componentes enfocados en presentación y coordinación de UI, y servicios enfocados en capacidades compartidas.

Aplicación al proyecto. `RouteFiltersComponent` gestiona los filtros y `RouteTableComponent` presenta la tabla y la selección. `RoutesApiService` conoce los endpoints y tipos HTTP. Los componentes solicitan listar o ejecutar sin construir URLs ni repetir el manejo del transporte.

23. ¿Qué es RxJS y cómo se utiliza en Angular?

Respuesta. RxJS es una biblioteca de programación reactiva basada en observables. Permite trabajar con valores que llegan a lo largo del tiempo y transformarlos, combinarlos, cancelarlos o manejar sus errores mediante operadores. Angular la utiliza en `HttpClient`, formularios, navegación y eventos. Lo importante es construir flujos claros y controlar correctamente las suscripciones.

Aplicación al proyecto. Los filtros pueden emitir cambios, `debounceTime` evita llamadas por cada tecla y `switchMap` garantiza que una respuesta vieja no reemplace la búsqueda nueva. Para ejecutar rutas usaría `exhaustMap` si se debe impedir doble clic durante la petición.

24. ¿Cómo manejarías el estado de una aplicación Angular en un proyecto empresarial?

Respuesta. Clasificaría el estado antes de elegir una herramienta: local de componente, estado de URL, estado de servidor y estado compartido de una feature. Usaría signals para estado local y derivado, query params para filtros navegables y un servicio/facade por feature para coordinación. Incorporaría un store global solo cuando el flujo entre muchas pantallas, los efectos y la depuración justifiquen su coste.

Aplicación al proyecto. Mantendría selección visible en el componente o store de la feature, filtros/página en query params y datos/carga/error en `RoutesStore` o facade. Tras importar o ejecutar, invalidaría o actualizaría la lista explícitamente.

25. ¿Para qué sirven los guards en Angular?

Respuesta. Los guards controlan navegación: CanActivate protege entrada, CanActivateChild una rama, CanDeactivate evita abandonar una pantalla y perder cambios, y CanMatch decide si una ruta coincide o carga según una condición. Mejoran flujo y experiencia, pero nunca sustituyen autorización en el backend porque el cliente puede manipularse.

Aplicación al proyecto. Un guard puede ocultar la pantalla de importación o ejecución a un usuario lector y CanDeactivate podría proteger un formulario de creación modificado. La API vuelve a comprobar el permiso en cada solicitud.

26. ¿Cómo funcionan los formularios reactivos y cuándo usarlos?

Respuesta. En los formularios reactivos, la estructura del formulario se define en TypeScript mediante FormControl, FormGroup y FormArray. Esto permite controlar valores, estados y errores de forma explícita. Los usaría en formularios empresariales, dinámicos o con validaciones entre varios campos, porque facilitan las pruebas y ofrecen un comportamiento más predecible.

Aplicación al proyecto. La creación de una ruta requiere campos relacionados, coordenadas y fechas, por lo que un formulario reactivo tipado permite validadores por campo y de rango. Al enviarlo transforma el valor a un DTO y presenta los errores por campo devueltos por DRF.

27. ¿Qué son los interceptores HTTP y qué casos de uso tienen?

Respuesta. Son middleware de HttpClient que envuelven cada petición y el stream de respuesta. Centralizan comportamiento transversal como credenciales, correlación, timeouts, medición, normalización de errores o caché. En Angular actual preferiría interceptores funcionales por su orden más predecible. No deben esconder reglas de negocio ni reintentar operaciones no idempotentes sin protección.

Aplicación al proyecto. Añadiría autenticación y X-Request-ID, mediría latencia y convertiría errores de red en un tipo uniforme. El servicio conservaría los errores de validación por campo porque son parte del contrato, no un simple toast genérico.

28. ¿Cómo estructurarías una arquitectura Angular escalable para múltiples módulos?

Respuesta. Organizaría Angular por funcionalidades del negocio. Cada funcionalidad tendría sus páginas, componentes, servicios, modelos y rutas, procurando que sus dependencias sean claras. Usaría componentes standalone y carga diferida cuando aporte valor. Los elementos realmente comunes quedarían en core o shared, evitando convertir esas carpetas en lugares para código sin dueño.

Aplicación al proyecto. La pantalla actual puede evolucionar a features routes, imports y executions, cada una cargada por ruta. Tabla, estado vacío y controles del dominio permanecen dentro de routes; botones genéricos y layout van al sistema de UI solo si existe reutilización real.